Enterprise Airline Reservation System

Relational Database Schema Design

Case Study 1 — Group B10

Student Names	Roll Numbers
Swapnil Patil	25bcs10619
Shiven Kathuria	25bcs10290
Krishna Shukla	25bcs10094
Javin Arora	25bcs10369
Arjun Saxena	25bcs10099
Karukonda Vamshi Krishna	vamshi.25bcs10043
Manasvi Lakkireddy	25bcs10760
Pathan Rizwan Ahmad Khan	25bcs10266
Priyadarshini Jaliparthi	25bcs10781
Harshal Chandak	25bcs10732

1 Assumptions

The following assumptions were made in interpreting the problem statement and shaping all subsequent design decisions — from entity identification through to workflow definition and architectural choices. They are grouped by the domain they govern.

1.1 Hierarchical Transactions

The booking model operates across two distinct layers. `pnr_group` functions as the order envelope — it holds the passenger-facing PNR code, contact snapshot, and payment reference. `booking` functions as the line item — one row per passenger per flight leg. A family of four on a connecting itinerary (BOM→DEL→LHR) produces eight `booking` rows under a single `pnr_id`.

This split makes segment-level rescheduling non-destructive. Rescheduling one leg inserts a new `booking` row with `rescheduled_from_id` pointing to the replaced segment; all other rows in the PNR are untouched.

Accepted limitation: Payment responsibility is modelled at the PNR level, not the passenger level. Split-payment scenarios (each traveller in a group paying separately) are out of scope for this case study and would require moving the `payment` FK from `pnr_id` to `booking_id`.

1.2 Data Integrity & Snapshots

Two attributes intentionally snapshot live data at the moment of transaction rather than referencing master records:

- **`pnr_group.contact_email` / `contact_phone`** — copied from the passenger profile at purchase time. If the passenger later updates their profile, historical receipts and dispute records remain tied to the contact data that was live at transaction completion.
- **`baggage.excess_fee`** — calculated at check-in against the fare class allowance threshold and stored as a static `DECIMAL`. If the airline subsequently changes the allowance policy, historical fee records remain legally accurate without retroactive distortion.

Both fields are treated as write-once after their respective transaction events. This is an application-layer contract — no database constraint enforces immutability post-commit.

1.3 Performance Strategy — CQRS

BCNF normalisation minimises write overhead but introduces deep join chains on the read path. A single passenger dashboard call spans approximately seven to eight tables. Executing that join on every request at scale is not viable.

The CQRS pattern resolves this by decoupling write and read concerns entirely:

- **Write path:** All mutations hit the normalised MySQL instance. ACID guarantees and FK integrity are enforced here.
- **Event pipeline:** On commit, the application fires an event onto the message bus (Kafka or RabbitMQ). The API response returns immediately — no synchronous dependency on downstream updates.
- **Read path:** Event consumers execute the expensive join once, then push the result into Redis or Elasticsearch as a pre-joined document. All subsequent reads are single-key lookups with zero joins.

Accepted trade-off: A brief read-after-write inconsistency window exists between MySQL commit and Redis cache update. The write transaction remains the single source of truth. Cache invalidation is an application-layer responsibility — the schema has no mechanism to enforce cache coherence.

1.4 Delay Records as a 1:M Relationship

A single flight can experience multiple discrete delay events in sequence — for example, an initial ATC ground stop followed by a weather extension and a subsequent crew rest violation. Storing delay data directly on the `flight` row would require either overwriting prior events (losing history) or concatenating values (violating normalisation).

`delay_record` is therefore a 1:M child of `flight`. Each row captures one delay event with its own reason code, duration, and `notification_sent` flag. This flag allows the notification service to query only unalerted delay events, preventing duplicate passenger communications across a multi-event delay chain.

Clarification on `delay_minutes` semantics**:** Each row stores the duration of that specific delay event, not a cumulative total. Total flight delay is derived via `SUM(delay_minutes)` across all child rows — either at query time or pre-computed in the CQRS read store.

2 ER Diagram

The ERD was designed using Mermaid.js and rendered as an interactive HTML document. Access it via the link below:

→ View ERD: Airline ERD

The diagram covers all 22 tables across 7 functional domains, with relationship cardinality and foreign key lines annotated. The ERD file is also submitted alongside this document as `ERD.html`.

3 Relational Data Dictionary — Table Specifications

Group A — Customer Profiles & Loyalty

Table 1: loyalty_tier

loyalty_tier — *Defines point boundaries and benefit levels for the frequent flyer programme*

Attribute	Data Type	Key	Null?	Constraints / Defaults	Reasoning
tier_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate primary key; auto-assigned on insert
tier_name	`VARCHAR(30)`	UQ	NOT NULL	CHECK IN ('Classic','Silver','Gold','Platinum')	Enumerated tier labels; unique constraint prevents duplicate tier names
min_points	`INT`	—	NOT NULL	CHECK (min_points >= 0)	Lower boundary of point range for this tier
max_points	`INT`	—	NULL	—	NULL for the top-tier (Platinum) which has no upper bound
benefits_description	`TEXT`	—	NULL	—	Free-text description; not operationally queried so no index needed

Table 2: passenger

passenger — *Maintains identity records and account profiles for individual travellers*

Attribute	Data Type	Key	Null?	Constraints / Defaults	Reasoning
passenger_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate key; all booking references point here
full_name	`VARCHAR(100)`	—	NOT NULL	—	Legal name as per travel document

passport_number	VARCHAR(30)	UQ	NOT NULL	UNIQUE	Global identity key; unique constraint blocks duplicate registration
nationality	CHAR(3)	—	NOT NULL	ISO Alpha-3 country code	Stored as 3-char code to avoid transitive dependency on country_name
date_of_birth	DATE	—	NOT NULL	—	Required for age-verification and international documentation
email	VARCHAR(120)	UQ	NOT NULL	UNIQUE	Primary login vector; unique ensures one account per email
phone	VARCHAR(20)	—	NULL	—	Optional; contact captured at PNR level for transactional comms
frequent_flyer_number	VARCHAR(20)	UQ	NULL	UNIQUE; CHECK (frequent_flyer_number IS NULL OR CHAR_LENGTH(frequent_flyer_number) >= 8)	NULL = non-member; assigned only when passenger joins loyalty programme
loyalty_tier_id	INT	FK	NULL	REFERENCES loyalty_tier(tier_id); NULL = non-member	Nullable: enforcing NOT NULL would break guest/non-enrolled profiles

Group B — Geographic Assets & Route Lines

Table 3: country

country — *Isolates geopolitical identifiers; eliminates transitive dependency from city assets*

Attribute	Data Type	Key	Null?	Constraints / Defaults	Reasoning
country_code	CHAR(3)	PK	NOT NULL	ISO Alpha-3; unique standalone key	BCNF fix: country_name was transitively dependent on country_code, not city_id
country_name	VARCHAR(80)	UQ	NOT NULL	UNIQUE	Unique constraint prevents two rows with the same country name

Table 4: city

city — *Defines municipal hubs, linked to their sovereign country*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
city_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate key for join efficiency
city_name	`VARCHAR(80)`	—	NOT NULL	—	Municipal name; no uniqueness constraint (cities share names across countries)
country_code	`CHAR(3)`	FK	NOT NULL	REFERENCES country(country_code)	Direct FK to country; satisfies BCNF by removing the transitive path

Table 5: airport

airport — *Defines operational stations with IATA/ICAO codes and timezone data*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
airport_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate key referenced by route table
iata_code	`CHAR(3)`	UQ	NOT NULL	UNIQUE; global 3-letter standard	Used in all passenger-facing references (boarding passes, search)
icao_code	`CHAR(4)`	UQ	NOT NULL	UNIQUE; global 4-letter ATC standard	Used in air traffic control and crew scheduling systems
airport_name	`VARCHAR(120)`	—	NOT NULL	—	Display name for UI and reports
city_id	`INT`	FK	NOT NULL	REFERENCES city(city_id)	Links airport to its city for geographic hierarchy traversal
timezone	`VARCHAR(40)`	—	NOT NULL	IANA TZ format e.g. 'Asia/Kolkata'	Critical for converting UTC scheduled times to local display times

Table 6: route

route — *Abstract flight corridors connecting two airport stations*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
route_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate key; referenced by flight instances
origin_airport_id	`INT`	FK	NOT NULL	REFERENCES airport(airport_id)	Origin station; no self-reference check needed (handled at app layer)
destination_airport_id	`INT`	FK	NOT NULL	REFERENCES airport(airport_id); UNIQUE(origin_airport_id, destination_airport_id)	Composite unique prevents duplicate route corridors

distance_km	`INT`	—	NOT NULL	CHECK (distance_km > 0)	Stored once on the route; never repeated on individual flight rows

Group C — Fleet Configuration & Scheduling

Table 7: aircraft_model

aircraft_model — *Isolates technical blueprints from physical hull records*

Attribute	Data Type	Key	Null?	Constraints / Defaults	Reasoning
model_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Shared by all physical hulls of the same type
model_name	`VARCHAR(60)`	—	NOT NULL	—	E.g. 'A320neo', '737 MAX 8'
manufacturer	`VARCHAR(60)`	—	NOT NULL	—	E.g. 'Airbus', 'Boeing'
total_seat_capacity	`INT`	—	NOT NULL	CHECK (total_seat_capacity > 0)	Blueprint capacity; actual seat rows are per-flight in seat table
seat_layout_description	`TEXT`	—	NULL	—	Optional descriptor; not queried operationally

Table 8: aircraft

aircraft — *Tracks individual physical hulls with runtime status and registration data*

Attribute	Data Type	Key	Null?	Constraints / Defaults	Reasoning
aircraft_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate key for scheduling joins
tail_number	`VARCHAR(15)`	UQ	NOT NULL	UNIQUE; global hull registration standard	Unique physical identifier stamped on the airframe
model_id	`INT`	FK	NOT NULL	REFERENCES aircraft_model(model_id)	Points to blueprint; avoids repeating capacity/manufacturer per hull
manufacture_year	`INT`	—	NOT NULL	—	Used for maintenance lifecycle tracking
status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('active','maintenance','retired')	Prevents scheduling of retired or grounded aircraft

Table 9: flight

flight — *Binds a route and aircraft to a specific scheduled departure instance*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
flight_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Referenced by booking, seat, crew_assignment, delay_record
flight_number	`VARCHAR(10)`	—	NOT NULL	UNIQUE(flight_number, scheduled_departure)	Composite unique: same flight number reused daily but unique per departure
route_id	`INT`	FK	NOT NULL	REFERENCES route(route_id)	Carries origin/destination and distance; not repeated on flight row
aircraft_id	`INT`	FK	NOT NULL	REFERENCES aircraft(aircraft_id)	Determines physical capacity and model for this instance
scheduled_departure	`DATETIME`	—	NOT NULL	Stored in UTC	App layer converts to local time using airport.timezone
scheduled_arrival	`DATETIME`	—	NOT NULL	Stored in UTC; CHECK (scheduled_arrival > scheduled_departure)	Arrival must be after departure; enforced by CHECK constraint
status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('scheduled','boarding','departed','arrived','cancelled')	Operational state; mutated directly (unlike booking which uses ledger)
gate	`VARCHAR(10)`	—	NULL	—	Assigned closer to departure; NULL until gate is confirmed
terminal	`VARCHAR(10)`	—	NULL	—	Same as gate; assigned dynamically

The composite unique on (flight_number, scheduled_departure) assumes that a given flight number operates at most once per departure timestamp.

Group D — Inventory Layout & Fare Classes

Table 10: fare_class

fare_class — *Manages cabin policy rules, baggage allowances, and service definitions*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
fare_class_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Referenced by seat and booking tables
class_name	`VARCHAR(20)`	UQ	NOT NULL	CHECK IN ('economy','business','first'); UNIQUE	Enumerated set; uniqueness prevents duplicate class definitions
baggage_allowance_kg	`INT`	—	NOT NULL	—	Base allowance threshold; excess computed at ground check-in

refund_policy	TEXT	—	NULL	—	Free text; varies by promotional booking context
change_policy	TEXT	—	NULL	—	Free text; varies by ticket type
meal_service	TINYINT(1)	—	NOT NULL	DEFAULT 0	MySQL boolean substitute; 1 = meal included, 0 = no meal

Table 11: seat

seat — *Tracks seat allocation matrix per discrete flight session*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
seat_id	INT	PK	NOT NULL	AUTO_INCREMENT	Unique per seat-flight combination
flight_id	INT	FK	NOT NULL	REFERENCES flight(flight_id)	Scopes seat to a specific flight instance, not the aircraft model
seat_number	VARCHAR(5)	—	NOT NULL	UNIQUE(flight_id, seat_number); e.g. '14A'	Composite unique: same seat number valid on different flights
fare_class_id	INT	FK	NOT NULL	REFERENCES fare_class(fare_class_id)	Determines which cabin class rules apply to this physical seat
availability_status	VARCHAR(20)	—	NOT NULL	CHECK IN ('available','held','booked')	Operational state; 'held' used during active checkout sessions
hold_expires_at	DATETIME	—	NULL	—	Session lock expiry; NULL when seat is available or permanently booked
seat_features	VARCHAR(20)	—	NULL	CHECK IN ('window','aisle','middle','extra_legroom')	Physical attribute used for seat selection UI filters

Group E — Core Order Layers & Financial Tracking

Table 12: pnr_group

pnr_group — *Parent transactional envelope — groups multi-passenger, multi-leg itineraries*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
pnr_id	INT	PK	NOT NULL	AUTO_INCREMENT	Root key; all bookings and payments reference this
pnr_code	CHAR(6)	UQ	NOT NULL	UNIQUE; globally unique 6-character alphanumeric	Passenger-facing reference code; indexed for O(log n) lookup
lead_passenger_id	INT	FK	NULL	REFERENCES passenger(passenger_id)	Optional link to registered profile; NULL for guest checkouts
contact_email	VARCHAR(120)	—	NOT NULL	—	Text snapshot at time of purchase; used for receipts/notifications

contact_phone	VARCHAR(20)	—	NOT NULL	—	Snapshot at purchase; NOT NULL as SMS dispatch requires a number
created_at	DATETIME	—	NOT NULL	DEFAULT CURRENT_TIMESTAMP	Booking creation time in UTC

Table 13: booking

booking — *Maps one passenger to one flight segment; each row is an itinerary line item*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
booking_id	INT	PK	NOT NULL	AUTO_INCREMENT	Granular segment key; self-referenced for rescheduling lineage
pnr_id	INT	FK	NOT NULL	REFERENCES pnr_group(pnr_id) ON DELETE CASCADE	Parent order envelope; cascade delete cleans up orphan line items
passenger_id	INT	FK	NOT NULL	REFERENCES passenger(passenger_id)	Individual traveller for this segment
flight_id	INT	FK	NOT NULL	REFERENCES flight(flight_id)	Specific flight instance booked
seat_id	INT	FK	NOT NULL	REFERENCES seat(seat_id)	Exact seat assigned; status updated to 'booked' on confirm
fare_class_id	INT	FK	NOT NULL	REFERENCES fare_class(fare_class_id)	Snapshot of class at time of booking; decoupled from seat's class
booking_datetime	DATETIME	—	NOT NULL	DEFAULT CURRENT_TIMESTAMP	Exact transaction timestamp in UTC
ticket_number	VARCHAR(20)	UQ	NOT NULL	UNIQUE; 13-digit commercial e-ticket string	Airline industry standard; globally unique per segment
source_channel	VARCHAR(20)	—	NOT NULL	CHECK IN ('vistara_web','vistara_app', 'ota_partner','gds')	Tracks acquisition channel for revenue analytics
external_booking_ref	VARCHAR(50)	—	NULL	—	OTA or GDS cross-reference code; NULL for direct bookings
rescheduled_from_id	INT	FK	NULL	REFERENCES booking(booking_id); self-referential	Unary FK; links to the original segment this row replaced

Table 14: booking_status

booking_status — *Append-only audit ledger for booking lifecycle state transitions*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
status_id	INT	PK	NOT NULL	AUTO_INCREMENT	Ledger row key; never updated, only inserted
booking_id	INT	FK	NOT NULL	REFERENCES booking(booking_id) ON DELETE CASCADE	Parent booking segment
status	VARCHAR(20)	—	NOT NULL	CHECK IN ('confirmed','cancelled','rescheduled','waitlisted','no_show')	Added 'no_show' to align with checkin.boarding_status operational outcome
changed_at	DATETIME	—	NOT NULL	DEFAULT CURRENT_TIMESTAMP	Exact timestamp of state transition
changed_reason	TEXT	—	NULL	—	Explanatory string for operational and audit review

Table 15: payment

payment — *Logs aggregated transaction records at the PNR order level*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
payment_id	INT	PK	NOT NULL	AUTO_INCREMENT	Unique transaction key
pnr_id	INT	FK	NOT NULL	REFERENCES pnr_group(pnr_id) ON DELETE CASCADE	One payment per PNR order; billing is aggregated, not per-passenger
amount	DECIMAL(10,2)	—	NOT NULL	CHECK (amount > 0)	Fixed CHECK threshold: changed from '> 10' to '> 0' to support low-value bookings
currency	CHAR(3)	—	NOT NULL	DEFAULT 'INR'	ISO 4217 currency code; exchange rate not stored (gap — see design notes)
payment_method	VARCHAR(20)	—	NOT NULL	CHECK IN ('card','upi','netbanking','wallet','ota_collected')	Enumerated payment channels for reconciliation
payment_status	VARCHAR(20)	—	NOT NULL	CHECK IN ('pending','completed','failed')	Current state; no ledger needed as payment is immutable post-completion
transaction_datetime	DATETIME	—	NOT NULL	—	UTC timestamp of gateway response
exchange_rate_to_inr	DECIMAL(10, 4)	—	NULL	—	Populated at transaction time for foreign currency payments

Table 16: refund

refund — *Tracks reversal amounts issued against specific payment transactions*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
refund_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Each refund action is a separate row (supports partial refunds)
payment_id	`INT`	FK	NOT NULL	REFERENCES payment(payment_id) ON DELETE CASCADE	Multiple refund rows can exist per payment for partial cancellations
refund_amount	`DECIMAL(10,2)`	—	NOT NULL	CHECK (refund_amount > 0)	Positive value; direction (credit) implied by context
refund_status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('pending','processed','rejected')	Lifecycle state of the reversal action
refund_datetime	`DATETIME`	—	NULL	—	NULL until refund is processed by payment gateway
refund_reason	`TEXT`	—	NULL	—	Operational notes; required for accounting audit trail

Group F — Ground Logistics & Operational States

Table 17: checkin

checkin — *Tracks security verification and terminal access per individual flight segment*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
checkin_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Unique check-in event key
booking_id	`INT`	FK, UQ	NOT NULL	REFERENCES booking(booking_id) ON DELETE CASCADE; UNIQUE	Unique constraint enforces exactly one check-in record per leg
checkin_datetime	`DATETIME`	—	NOT NULL	—	Timestamp of check-in action in UTC
checkin_channel	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('web','kiosk','counter','mobile')	Channel analytics for operations reporting
boarding_pass_number	`VARCHAR(30)`	UQ	NOT NULL	UNIQUE	Globally unique per boarding event; used at gate scanners
boarding_status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('checked_in','boarded','no_show')	Gate-level status; 'no_show' propagated to booking_status ledger
boarding_time	`DATETIME`	—	NULL	—	Exact gate passage timestamp; NULL until physically scanned

gate_used	`VARCHAR(10)`	—	NULL	—	Actual gate at boarding; may differ from flight.gate if reassigned

Table 18: baggage

baggage — *Tracks registered luggage items per passenger flight segment*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
baggage_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	One row per bag tag
booking_id	`INT`	FK	NOT NULL	REFERENCES booking(booking_id) ON DELETE CASCADE	A booking can have multiple bags
bag_tag_number	`VARCHAR(30)`	UQ	NOT NULL	UNIQUE; universal barcode standard	Scanned at every baggage handling touchpoint
weight_kg	`DECIMAL(5,2)`	—	NOT NULL	CHECK (weight_kg > 0)	Measured at check-in counter scale
baggage_type	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('checked','carry_on','oversized')	Drives handling routing and excess fee calculation
status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('checked_in','loaded','delivered','lost')	Lifecycle state; tracked by baggage handling system
excess_fee	`DECIMAL(8,2)`	—	NULL	—	Static snapshot at check-in; protects historical ledger from fare-class policy changes

Table 19: special_service_request

special_service_request — *Captures ancillary services and accessibility provisions per leg segment*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
ssr_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	One row per service request
booking_id	`INT`	FK	NOT NULL	REFERENCES booking(booking_id) ON DELETE CASCADE	Multiple SSRs allowed per booking
service_type	`VARCHAR(30)`	—	NOT NULL	CHECK IN ('wheelchair','infant_cot','meal_preference','medical','unaccompanied_minor')	Standard IATA SSR codes
service_details	`TEXT`	—	NULL	—	Granular notes e.g. specific meal type, allergy information
fulfilment_status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('requested','confirmed','fulfilled','denied')	Tracks whether ground operations acted on the request

Group G — Flight Operations & Crew Telemetry

Table 20: crew

crew — *Stores reference metadata and certification indexes for flight operations personnel*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
crew_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Referenced by crew_assignment junction table
full_name	`VARCHAR(100)`	—	NOT NULL	—	Legal name per civil aviation registry
licence_number	`VARCHAR(30)`	UQ	NOT NULL	UNIQUE; civil aviation registry key	Globally unique licence; used for regulatory compliance checks
role	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('pilot','co_pilot','cabin_crew','purser')	Determines minimum crew count validation at assignment layer
availability_status	`VARCHAR(20)`	—	NOT NULL	CHECK IN ('available','on_duty','on_leave')	Checked before any new assignment is committed

Table 21: crew_assignment

crew_assignment — *Junction table resolving M:N relationship between crew and flight*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
assignment_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	Surrogate key for the junction row
flight_id	`INT`	FK	NOT NULL	REFERENCES flight(flight_id) ON DELETE CASCADE	Target flight for this duty assignment
crew_id	`INT`	FK	NOT NULL	REFERENCES crew(crew_id); UNIQUE(flight_id, crew_id)	Composite unique: one crew member assigned to a flight at most once
assigned_role	`VARCHAR(30)`	—	NOT NULL	—	Specific role on this flight; can differ from crew.role (e.g. acting purser)
assignment_datetime	`DATETIME`	—	NOT NULL	—	Scheduling record timestamp; used with idx_crew_assignment_schedule
duty_hours_logged	`DECIMAL(4,2)`	—	NULL	—	Populated post-flight by ops system; NULL until flight completes

Table 22: delay_record

delay_record — *Tracks active flight disruptions; separated from core scheduling records*					
Attribute	**Data Type**	**Key**	**Null?**	**Constraints / Defaults**	**Reasoning**
delay_id	`INT`	PK	NOT NULL	AUTO_INCREMENT	One row per delay event; relationship updated to 1:M (see design notes)
flight_id	`INT`	FK	NOT NULL	REFERENCES flight(flight_id) ON DELETE CASCADE	Parent flight; multiple delay events allowed per flight
delay_reason_code	`VARCHAR(10)`	—	NOT NULL	Standardised IATA delay reason codes	E.g. 'ATC', 'WX', 'MX' for air traffic, weather, maintenance
delay_minutes	`INT`	—	NOT NULL	CHECK (delay_minutes > 0)	Changed from > 10 to > 0; any recordable delay is operationally meaningful
actual_departure	`DATETIME`	—	NULL	—	Recorded by departure control system post-pushback
actual_arrival	`DATETIME`	—	NULL	—	Recorded on gate-in confirmation
notification_sent	`TINYINT(1)`	—	NOT NULL	DEFAULT 0	Flag checked by notification service; 1 = passengers already alerted

4 Database Index Specifications

4.1 Custom Performance Indexes

Index Name	Target Table & Columns	Index Type	Why We Use It
idx_flight_route_date	`flight` `(route_id, scheduled_departure)`	B-Tree Composite	• Targets the most frequent search query: flights between two airports on a given date. • Composite key allows the engine to equality-match on route_id, then range-scan over scheduled_departure within that corridor. • Without this index, every date-range flight search would trigger a full table scan of the entire flight schedule. • Column order is intentional: low-cardinality route_id first to maximise selectivity before the timestamp range scan.
idx_seat_inventory_lookup	`seat` `(flight_id, availability_status, fare_class_id, seat_number)`	B-Tree Composite (Covering)	• Targets the seat availability query: available seats by fare class for a selected flight. • MySQL InnoDB does not support INCLUDE columns; all four fields are added to the index key directly. • Placing fare_class_id and seat_number in the index allows the query to be answered entirely from the index B-Tree without a row lookup (covering index effect). • Prevents row-locking contention during high-traffic checkout windows by eliminating heap fetches.
idx_checkin_boarding_alert	`checkin` `(boarding_status, booking_id)`	B-Tree Composite (Full Index — MySQL limitation)	• Targets the gate operations query: passengers checked in but not yet boarded. • MySQL does not support partial/filtered indexes with WHERE clauses; this is a full B-Tree index. • Application layer adds WHERE boarding_status = 'checked_in' at query time; index prefix on boarding_status still enables fast selective scans. • booking_id is included as a suffix so gate agents can retrieve booking details without a secondary lookup. • Design note: in PostgreSQL this would be a filtered partial index; in MySQL the same selectivity is achieved via application-layer query filtering.

Index Name	Target Table & Columns	Index Type	Why We Use It
idx_pnr_code	`pnr_group` `(pnr_code)`	B-Tree UNIQUE (replaces Hash — MySQL limitation)	• Targets the exact-match lookup query: passenger retrieving their itinerary by PNR code. • MySQL InnoDB does not support explicit hash indexes; a UNIQUE B-Tree index is used instead. • InnoDB automatically builds an adaptive hash index in memory on hot B-Tree pages, approximating O(1) lookup for repeated access patterns. • The UNIQUE constraint also enforces the business rule that PNR codes are globally non-repeating. • Design fix from original schema: UNIQUE HASH on PostgreSQL was contradictory; hash indexes cannot enforce uniqueness. MySQL's B-Tree UNIQUE cleanly solves both concerns.
idx_flight_delay_tracking	`delay_record` `(delay_minutes, flight_id)`	B-Tree Composite (Full Index — MySQL limitation)	• Targets the operations dashboard query: flights delayed beyond 2 hours (120 minutes). • MySQL does not support WHERE-clause filtered indexes; full index on delay_minutes is used. • Application query adds WHERE delay_minutes > 120; the B-Tree index still enables efficient range scans from 120 upward. • flight_id as a suffix allows dashboard queries to retrieve affected flight IDs directly from the index without a row lookup. • Design note: CHECK (delay_minutes > 0) on the table and the > 120 query filter are separate concerns — the CHECK enforces data validity; the index serves the specific dashboard threshold.
idx_crew_assignment_schedule	`crew_assignment` `(crew_id, assignment_datetime)`	B-Tree Composite	• Targets the scheduling conflict check: a crew member cannot be assigned to more than one flight per day. • Composite key allows the engine to equality-match on crew_id, then range-scan the assignment_datetime for overlapping duty windows. • This index is hit on every new crew assignment write, making it the most critical index on the write path. • Without it, detecting scheduling conflicts would require a full scan of all assignments for that crew member.

4.2 Index Build Script (MySQL 8.0)

Note: UNIQUE and FK indexes are auto-generated by MySQL. The script below covers only custom performance indexes.

```
-- Index 1: Route + date composite for flight search
CREATE INDEX idx_flight_route_date
  ON flight (route_id, scheduled_departure);

-- Index 2: Covering index for seat inventory checkout
CREATE INDEX idx_seat_inventory_lookup
  ON seat (flight_id, availability_status, fare_class_id, seat_number);

-- Index 3: Gate boarding status (full index; app filters WHERE boarding_status = 'checked_in')
CREATE INDEX idx_checkin_boarding_alert
  ON checkin (boarding_status, booking_id);

-- Index 4: PNR code lookup (B-Tree UNIQUE; replaces unsupported hash index)
-- Note: UNIQUE constraint on pnr_group(pnr_code) already creates this automatically.
--       Explicit creation only needed if constraint was not declared.
CREATE UNIQUE INDEX idx_pnr_code
  ON pnr_group (pnr_code);

-- Index 5: Delay minutes range scan (full index; app filters WHERE delay_minutes > 120)
CREATE INDEX idx_flight_delay_tracking
  ON delay_record (delay_minutes, flight_id);

-- Index 6: Crew scheduling conflict detection
CREATE INDEX idx_crew_assignment_schedule
  ON crew_assignment (crew_id, assignment_datetime);
```

5 Normalisation Choices & High-Level Architecture

5.1 Target Normalisation Level — Strict BCNF

The schema operates at Boyce-Codd Normal Form (BCNF) throughout. A relation is in BCNF if, for every non-trivial functional dependency X → Y, the determinant X is a super-key. This eliminates all transitive and partial dependencies tolerated by lower forms (2NF, 3NF).

Key BCNF Enforcement Decisions

- **Geographic Hierarchy (Country → City → Airport):** Storing country_name inside City creates a transitive dependency: city_id → country_code → country_name. Resolved by extracting a standalone Country table where country_code is the sole determinant of country_name.
- **Fleet Configuration (AircraftModel → Aircraft):** Storing capacity and manufacturer on individual hull records creates functional redundancy — model attributes depend on model_id, not tail_number. Resolved by isolating blueprints in AircraftModel and pointing Aircraft rows to model_id.
- **Loyalty Programme Split (LoyaltyTier → Passenger):** Mixing tier rules with passenger identity data risks deletion anomalies: removing the last passenger on a tier would destroy the tier's point boundaries. Resolved by separating tiers into their own table with loyalty_tier_id as a nullable FK on Passenger.

5.2 Controlled Denormalisation — Justified Exceptions

Two attributes intentionally deviate from strict BCNF for financial and operational integrity reasons:

- **baggage.excess_fee:** Calculated from weight vs. fare_class baggage_allowance_kg. Stored as a static snapshot at check-in time. If the FareClass allowance threshold changes in the future, historical excess fee calculations remain accurate and legally auditable.
- **pnr_group.contact_email / contact_phone:** Passenger contact details are snapshotted at purchase time. The passenger may update their profile later, but the PNR retains the text that was live at transaction completion — critical for dispute resolution and receipt archival.

5.3 Outlier Table — booking_status (Below BCNF by Design)

The `booking_status` table is deliberately structured as an append-only time-series ledger rather than a normalised entity. Each row captures a point-in-time state transition with a timestamp and reason. This violates the spirit of BCNF (the same booking_id appears in multiple rows with different status values) but is the correct design choice for the following reasons:

- Maintains a complete, tamper-evident audit trail of all booking state changes.
- Prevents destructive overwrites — the 'cancelled' status is never written into the original 'confirmed' row.
- Enables time-travel queries: reconstruct the exact state of any booking at any historical point in time.
- Required for aviation regulatory compliance — cancellation and rebooking histories must be preserved.

5.4 High-Level Architecture — CQRS + Asynchronous Event Pipeline

Strict BCNF normalization is ideal for write integrity but creates deep join overhead on read queries. A Command Query Responsibility Segregation (CQRS) architecture decouples these concerns entirely.

Write Path (MySQL — Source of Truth)

- All data modifications (bookings, check-ins, seat updates, crew assignments) hit the normalised MySQL instance.
- Queries are narrow and targeted — BCNF splits mean each write touches a minimal number of table blocks.
- MySQL enforces ACID constraints: transactions are atomic, referential integrity is guaranteed by FK checks.
- The transactional database does zero heavy lifting for read-heavy use cases like flight search or dashboard analytics.

Asynchronous Event Pipeline (RabbitMQ / Apache Kafka)

- On successful transaction commit, the application layer fires an event payload onto the message bus (e.g. BookingConfirmedEvent, FlightDelayEvent).
- The API response is immediately returned to the client — no synchronous dependency on downstream read-store updates.
- Event consumers pick up payloads asynchronously, execute the expensive multi-table join exactly once, and push results to the read store.
- This decoupling means a brief read-store lag is acceptable; the write transaction is the single source of truth.

Read Path (Redis / Elasticsearch / Materialised Views)

- Pre-joined, denormalised JSON documents are cached in Redis keyed by PNR code, flight_id, or route+date.
- Passenger dashboard calls (GET itinerary by PNR) resolve as single-key point lookups — zero joins at read time.
- Flight search queries (seats available by fare class on a route) are served from Elasticsearch pre-computed indexes.
- Analytical queries (monthly route revenue, delay statistics) are served from materialised views refreshed by event consumers.

GDS Integration Layer

- The booking.source_channel field captures whether a booking originated from a GDS platform (Amadeus, Sabre, Travelport).
- booking.external_booking_ref stores the GDS-assigned PNR cross-reference for interline ticketing reconciliation.
- In production, a GDS adapter service sits between the GDS message bus (EDIFACT / NDC API) and the Booking Service, normalising external payloads into the internal schema format before writing to MySQL.
- The CQRS read store can expose a separate GDS-facing API endpoint that serves pre-joined itinerary payloads without exposing internal surrogate keys.

4.5 Closeness to Production Airline Booking Systems

This schema replicates several patterns used in modern airline PSS (Passenger Service Systems) such as Amadeus Altéa and Navitaire NewSkies:

- **PNR as order envelope:** Mirrors the industry standard where a PNR is the root transaction container, not the ticket. IATA NDC Level 4 uses the same parent order / order item structure as pnr_group → booking.
- **Append-only booking status ledger:** Identical to the event sourcing pattern used by Delta, Lufthansa, and Air India's reservation platforms — state changes are events, not mutations.
- **Seat inventory scoped per flight instance:** Mirrors Amadeus's seat map architecture where inventory is session-bound, not statically tied to the aircraft model.
- **CQRS + event bus:** Standard pattern in all major airline IT modernisation programs post-2015. Vistara's Salesforce + Amadeus integration follows a similar dual-store pattern.
- **Gaps from true production scale:** A production PSS would additionally include fare pricing tables with time-of-booking rates, codeshare flight modelling, ancillary product catalogues (upgrades, lounge access),

and IROPS (irregular operations) workflow tables. These are noted as known scope exclusions given the case study context.

5 Relationship Justifications

5.1 Many-to-Many (M:N) Relationships — Resolved via Junction Tables

Direct M:N relationships break First Normal Form due to multi-valued attributes and lead to severe data duplication. Both are decomposed into two 1:M relationships using junction tables:

Flight ↔ Crew → crew_assignment

- A single crew member serves multiple flights over a scheduling period.
- A single flight requires multiple crew members across different roles.
- The junction table captures assigned_role per flight instance — a pilot may act as co-pilot on a specific segment without changing their master Crew record.
- duty_hours_logged is a junction-level attribute — it belongs to the specific assignment, not to the crew member globally or the flight generically.
- UNIQUE(flight_id, crew_id) on the junction enforces the rule that one crew member cannot be double-assigned to the same flight.

Flight ↔ Seat (Physical Slot) → seat

- An aircraft model defines a static seat layout, but real-world availability is unique per flight instance.
- Mapping seat rows directly to flight_id (rather than aircraft_id) allows seat A12 to be 'available' on tomorrow's flight while 'booked' on today's.
- Session-level attributes (availability_status, hold_expires_at) live at the flight-seat intersection level, not on the aircraft blueprint.
- This isolates dynamic operational state from static hardware configuration — a core BCNF requirement.

5.2 One-to-Many (1:M) Relationships — Parent-Child Hierarchies

pnr_group → booking (Order Aggregation Layer)

- One PNR confirmation reference can contain multiple passengers on multiple flight legs.
- This 1:M structure supports family bookings, corporate group travel, and multi-leg connecting itineraries under one invoice.
- Contact information and payment details live on the parent PNR — not duplicated on every individual booking row.
- ON DELETE CASCADE ensures that if a PNR is cancelled and purged, all its child booking line items are cleaned up atomically.

country → city → airport (Geographic Hierarchy)

- Sovereign country contains multiple municipal cities; each city houses one or more operational airport stations.
- Strict hierarchical chain eliminates partial functional dependencies: airport_name does not determine country_name; city_id does not determine country_name directly.

- Renaming an airport, changing a city's classification, or updating a country code propagates correctly through FK references — no update anomalies.
- Airport.timezone is stored at the airport level because timezone boundaries do not always align with political city/country boundaries (e.g. Canberra vs Sydney).

airport → route → flight (Network Routing Hierarchy)

- An airport serves as the origin or destination of hundreds of unique route corridors.
- A single abstract route (e.g. BOM→DEL) is scheduled as hundreds of flight instances across a calendar year.
- Storing distance_km once on the route row — never repeated on individual flight rows — is a direct BCNF normalisation benefit.
- Separating route from flight allows analytics on route-level performance independent of specific flight instance disruptions.

pnr_group → payment → refund (Financial Ledger Chain)

- One PNR generates one aggregated gateway charge; billing is unified per party, not per passenger per leg.
- One payment can generate multiple refund actions — partial cancellation of a single traveller from a group of four produces one refund row, not a new payment record.
- This chain structure supports partial refund accounting without corrupting the original payment record.

booking → booking_status (Append-Only Audit Ledger)

- One booking accumulates multiple status rows over its lifecycle (confirmed → rescheduled → confirmed → cancelled).
- This 1:M structure is the only correct way to maintain a tamper-evident state history without destructive overwrites.
- Changed_at timestamps enable precise reconstruction of booking state at any historical moment for dispute resolution.

booking → checkin / baggage / special_service_request (Segment-Level Ground Ops)

- Ground events (check-in, baggage drop, SSR requests) happen at the individual flight leg level, not at the PNR order level.
- A passenger on a connecting itinerary (BOM→DEL→LHR) checks in separately for each leg; luggage is re-tagged at each hub.
- Tying these events to booking_id (not pnr_id) enables leg-specific tracking without conflating two-leg operations.

5.3 Unary / Self-Referential Relationship

booking.rescheduled_from_id → booking(booking_id)

- When a flight segment is rebooked due to weather, cancellation, or passenger request, the system must not destroy the original transaction.
- A new booking row is inserted with a rescheduled_from_id FK pointing back to the original booking_id.
- Positioned at the segment level (booking) rather than the PNR level: if only one leg of a three-leg itinerary is changed, the other two legs are unaffected.
- Supports multi-generation reschedule chains: A rescheduled to B, B rescheduled to C — each link preserved independently.
- NULLABLE: a standard first-time booking has no prior segment reference, so NULL correctly represents 'original booking'.